

TypeScript Interview Questions & Answers

Table of Contents

1. [Basic TypeScript Questions](#)
 2. [Type System Questions](#)
 3. [Advanced TypeScript Questions](#)
 4. [Practical Coding Questions](#)
 5. [Configuration & Tooling](#)
-

Basic TypeScript Questions

1. What is TypeScript and why should we use it?

Answer: TypeScript is a statically typed superset of JavaScript developed by Microsoft. It adds optional static typing to JavaScript and compiles to plain JavaScript.

Benefits:

- **Static Type Checking:** Catches errors at compile time
- **Better IDE Support:** Enhanced autocomplete, refactoring, and navigation
- **Code Documentation:** Types serve as documentation
- **Refactoring Safety:** Large codebases can be refactored with confidence
- **Modern JavaScript Features:** Supports latest ECMAScript features

```
typescript

// JavaScript
function greet(name) {
  return "Hello, " + name;
}

// TypeScript
function greet(name: string): string {
  return "Hello, " + name;
}
```

2. What are the basic data types in TypeScript?

Answer: TypeScript provides several built-in types:

typescript

// Primitive types

```
let isDone: boolean = false;
```

```
let decimal: number = 6;
```

```
let color: string = "blue";
```

// Array types

```
let list: number[] = [1, 2, 3];
```

```
let list2: Array<number> = [1, 2, 3];
```

// Tuple

```
let x: [string, number] = ["hello", 10];
```

// Enum

```
enum Color {Red, Green, Blue}
```

```
let c: Color = Color.Green;
```

// Any (avoid when possible)

```
let notSure: any = 4;
```

// Void

```
function warnUser(): void {  
    console.log("This is a warning message");  
}
```

// Null and Undefined

```
let u: undefined = undefined;
```

```
let n: null = null;
```

// Never (function that never returns)

```
function error(message: string): never {  
    throw new Error(message);  
}
```

3. What is the difference between `interface` and `type` in TypeScript?

Answer:

Interfaces:

- Can be extended and implemented
- Support declaration merging
- Better for object shapes

Types:

- More flexible (unions, intersections, primitives)
- Cannot be re-declared
- Better for computed types

typescript

```
// Interface
interface User {
  name: string;
  age: number;
}

interface User {
  email: string; // Declaration merging
}

interface Admin extends User {
  role: string;
}

// Type
type UserType = {
  name: string;
  age: number;
};

type AdminType = UserType & {
  role: string;
};

// Union types (only with type)
type Status = "loading" | "success" | "error";

// Computed types
type Keys = keyof UserType; // "name" | "age"
```

Type System Questions

4. Explain Generic Types in TypeScript

Answer: Generics allow you to create reusable components that work with multiple types while maintaining type safety.

typescript

// Generic function

```
function identity<T>(arg: T): T {  
    return arg;  
}
```

```
let output1 = identity<string>("myString");  
let output2 = identity<number>(100);
```

// Generic interface

```
interface GenericIdentityFn<T> {  
    (arg: T): T;  
}
```

// Generic class

```
class GenericNumber<T> {  
    zeroValue: T;  
    add: (x: T, y: T) => T;  
}
```

```
let myGenericNumber = new GenericNumber<number>();  
myGenericNumber.zeroValue = 0;  
myGenericNumber.add = function(x, y) { return x + y; };
```

// Generic constraints

```
interface Lengthwise {  
    length: number;  
}
```

```
function loggingIdentity<T extends Lengthwise>(arg: T): T {  
    console.log(arg.length); // Now we know it has a .length property  
    return arg;  
}
```

// Using keyof with generics

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {  
    return obj[key];  
}
```

```
let person = { name: "John", age: 30 };  
let name = getProperty(person, "name"); // string  
let age = getProperty(person, "age"); // number
```

5. What are Union and Intersection Types?

Answer:

Union Types (`|`): A value can be one of several types **Intersection Types** (`&`): A value must have all properties of multiple types

typescript

// Union Types

```
type StringOrNumber = string | number;
```

```
function formatValue(value: StringOrNumber): string {  
    if (typeof value === "string") {  
        return value.toUpperCase();  
    }  
    return value.toString();  
}
```

// Discriminated Unions

```
interface Bird {  
    type: "bird";  
    flyingSpeed: number;  
}
```

```
interface Horse {  
    type: "horse";  
    runningSpeed: number;  
}
```

```
type Animal = Bird | Horse;
```

```
function moveAnimal(animal: Animal) {  
    switch (animal.type) {  
        case "bird":  
            console.log("Flying at speed: " + animal.flyingSpeed);  
            break;  
        case "horse":  
            console.log("Running at speed: " + animal.runningSpeed);  
            break;  
    }  
}
```

// Intersection Types

```
interface BusinessPartner {  
    name: string;  
    credit: number;  
}
```

```
interface UserAccount {  
    email: string;  
    password: string;  
}
```

```
type Customer = BusinessPartner & UserAccount;
```

```
const customer: Customer = {  
  name: "John",  
  credit: 1000,  
  email: "john@example.com",  
  password: "secret"  
};
```

6. Explain Utility Types in TypeScript

Answer: TypeScript provides several utility types for common type transformations:

typescript

// Original interface

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  age: number;  
}
```

// Partial<T> - makes all properties optional

```
type PartialUser = Partial<User>;  
// { id?: number; name?: string; email?: string; age?: number; }
```

// Required<T> - makes all properties required

```
type RequiredUser = Required<PartialUser>;
```

// Pick<T, K> - picks specific properties

```
type UserContact = Pick<User, "name" | "email">;  
// { name: string; email: string; }
```

// Omit<T, K> - omits specific properties

```
type UserWithoutId = Omit<User, "id">;  
// { name: string; email: string; age: number; }
```

// Record<K, T> - creates object type with specific keys and values

```
type UserRoles = Record<string, "admin" | "user" | "guest">;  
// { [key: string]: "admin" | "user" | "guest"; }
```

// Readonly<T> - makes all properties readonly

```
type ReadonlyUser = Readonly<User>;
```

// ReturnType<T> - extracts return type of function

```
function getUser(): User {  
  return { id: 1, name: "John", email: "john@test.com", age: 30 };  
}
```

```
type UserReturnType = ReturnType<typeof getUser>; // User
```

// Parameters<T> - extracts parameter types

```
type UserParams = Parameters<typeof getUser>; // []
```

// Exclude<T, U> - excludes types

```
type StringOrNumberOrBoolean = string | number | boolean;  
type StringOrNumber = Exclude<StringOrNumberOrBoolean, boolean>; // string | number
```



```
// Extract<T, U> - extracts types
```

```
type StringType = Extract<StringOrNumberOrBoolean, string>; // string
```

Advanced TypeScript Questions

7. What are Mapped Types and how do you use them?

Answer: Mapped types allow you to create new types by transforming properties of existing types.

typescript

// Basic mapped type

```
type Readonly<T> = {  
  readonly [P in keyof T]: T[P];  
};
```

```
type Optional<T> = {  
  [P in keyof T]?: T[P];  
};
```

// Advanced mapped types

```
type Getters<T> = {  
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];  
};
```

```
interface Person {  
  name: string;  
  age: number;  
  location: string;  
}
```

```
type PersonGetters = Getters<Person>;  
// {  
//   getName: () => string;  
//   getAge: () => number;  
//   getLocation: () => string;  
// }
```

// Conditional mapped types

```
type NonNullable<T> = {  
  [P in keyof T]: T[P] extends null | undefined ? never : T[P];  
};
```

// Template literal types with mapped types

```
type EventHandlers<T> = {  
  [K in keyof T as `on${Capitalize<string & K>}Change`]: (value: T[K]) => void;  
};
```

```
type PersonEventHandlers = EventHandlers<Person>;  
// {  
//   onNameChange: (value: string) => void;  
//   onAgeChange: (value: number) => void;  
//   onLocationChange: (value: string) => void;  
// }
```

8. Explain Conditional Types

Answer: Conditional types select one of two possible types based on a condition expressed as a type relationship test.

```
typescript

// Basic conditional type
type IsString<T> = T extends string ? true : false;

type Test1 = IsString<string>; // true
type Test2 = IsString<number>; // false

// Practical example: NonNullable
type MyNonNullable<T> = T extends null | undefined ? never : T;

type Result1 = MyNonNullable<string | null>; // string
type Result2 = MyNonNullable<number | undefined>; // number

// Inferring with conditional types
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function greet(): string {
    return "Hello";
}

type GreetReturnType = ReturnType<typeof greet>; // string

// Distributive conditional types
type ToArray<T> = T extends any ? T[] : never;

type StringOrNumberArray = ToArray<string | number>; // string[] | number[]

// Complex conditional type
type FlattenArray<T> = T extends (infer U)[] ? U : T;

type Flattened1 = FlattenArray<string[]>; // string
type Flattened2 = FlattenArray<number>; // number

// Recursive conditional types
type DeepReadonly<T> = {
    readonly [P in keyof T]: T[P] extends object ? DeepReadonly<T[P]> : T[P];
};
```

9. How do you handle Error Handling in TypeScript?

Answer:

typescript

// Custom Error Classes

```
class ValidationError extends Error {
  constructor(public field: string, message: string) {
    super(message);
    this.name = "ValidationError";
  }
}

class NetworkError extends Error {
  constructor(public statusCode: number, message: string) {
    super(message);
    this.name = "NetworkError";
  }
}
```

// Result Pattern

```
type Result<T, E = Error> =
  | { success: true; data: T }
  | { success: false; error: E };

async function fetchUser(id: number): Promise<Result<User, NetworkError>> {
  try {
    const response = await fetch(`/api/users/${id}`);
    if (!response.ok) {
      return {
        success: false,
        error: new NetworkError(response.status, "Failed to fetch user")
      };
    }
    const user = await response.json();
    return { success: true, data: user };
  } catch (error) {
    return {
      success: false,
      error: new NetworkError(0, "Network request failed")
    };
  }
}
```

// Using the Result pattern

```
async function handleUser(id: number) {
  const result = await fetchUser(id);

  if (result.success) {
```

```

    console.log("User:", result.data.name);
  } else {
    console.error("Error:", result.error.message);
    if (result.error.statusCode === 404) {
      console.log("User not found");
    }
  }
}

// Option Pattern for null safety
type Option<T> = T | null;

function findUser(users: User[], id: number): Option<User> {
  return users.find(user => user.id === id) || null;
}

// Type Guards for Error Handling
function isValidError(error: unknown): error is ValidationError {
  return error instanceof ValidationError;
}

function isNetworkError(error: unknown): error is NetworkError {
  return error instanceof NetworkError;
}

try {
  // some operation
} catch (error) {
  if (isValidError(error)) {
    console.log(`Validation failed for field: ${error.field}`);
  } else if (isNetworkError(error)) {
    console.log(`Network error with status: ${error.statusCode}`);
  } else {
    console.log("Unknown error:", error);
  }
}

```

Practical Coding Questions

10. Implement a Generic Repository Pattern

Answer:

typescript

// Base Entity

```
interface BaseEntity {  
  id: number;  
  createdAt: Date;  
  updatedAt: Date;  
}
```

// Generic Repository Interface

```
interface IRepository<T extends BaseEntity> {  
  findById(id: number): Promise<T | null>;  
  findAll(): Promise<T[]>;  
  create(entity: Omit<T, keyof BaseEntity>): Promise<T>;  
  update(id: number, entity: Partial<Omit<T, keyof BaseEntity>>): Promise<T | null>;  
  delete(id: number): Promise<boolean>;  
  findBy(criteria: Partial<T>): Promise<T[]>;  
}
```

// Generic Repository Implementation

```
class Repository<T extends BaseEntity> implements IRepository<T> {  
  private data: T[] = [];  
  private nextId = 1;  
  
  async findById(id: number): Promise<T | null> {  
    const entity = this.data.find(item => item.id === id);  
    return entity || null;  
  }  
  
  async findAll(): Promise<T[]> {  
    return [...this.data];  
  }  
  
  async create(entity: Omit<T, keyof BaseEntity>): Promise<T> {  
    const now = new Date();  
    const newEntity = {  
      ...entity,  
      id: this.nextId++,  
      createdAt: now,  
      updatedAt: now  
    } as T;  
  
    this.data.push(newEntity);  
    return newEntity;  
  }  
}
```

```

    async update(id: number, entity: Partial<Omit<T, keyof BaseEntity>>): Promise<T | null> {
        const index = this.data.findIndex(item => item.id === id);
        if (index === -1) return null;

        const updatedEntity = {
            ...this.data[index],
            ...entity,
            updatedAt: new Date()
        } as T;

        this.data[index] = updatedEntity;
        return updatedEntity;
    }

    async delete(id: number): Promise<boolean> {
        const index = this.data.findIndex(item => item.id === id);
        if (index === -1) return false;

        this.data.splice(index, 1);
        return true;
    }

    async findBy(criteria: Partial<T>): Promise<T[]> {
        return this.data.filter(item => {
            return Object.keys(criteria).every(key => {
                return item[key as keyof T] === criteria[key as keyof T];
            });
        });
    }
}

```

// Usage Example

```

interface User extends BaseEntity {
    name: string;
    email: string;
    role: 'admin' | 'user';
}

interface Product extends BaseEntity {
    name: string;
    price: number;
    category: string;
}

```

// Create repositories

```

const userRepository = new Repository<User>();
const productRepository = new Repository<Product>();

```



```

// Usage
async function example() {
  // Create users
  const user1 = await userRepository.create({
    name: "John Doe",
    email: "john@example.com",
    role: "admin"
  });

  const user2 = await userRepository.create({
    name: "Jane Smith",
    email: "jane@example.com",
    role: "user"
  });

  // Find users
  const allUsers = await userRepository.findAll();
  const adminUsers = await userRepository.findBy({ role: "admin" });
  const specificUser = await userRepository.findById(1);

  // Update user
  const updatedUser = await userRepository.update(1, {
    name: "John Updated"
  });

  console.log("All users:", allUsers);
  console.log("Admin users:", adminUsers);
  console.log("Updated user:", updatedUser);
}

```

11. Implement a Type-Safe Event Emitter

Answer:

typescript

// Event map interface

```
interface EventMap {  
    [key: string]: any[];  
}
```

// Event emitter class

```
class TypeSafeEventEmitter<TEventMap extends EventMap> {  
    private listeners: {  
        [K in keyof TEventMap]?: Array<(...args: TEventMap[K]) => void>;  
    } = {};
```

// Add event listener

```
on<K extends keyof TEventMap>(  
    event: K,  
    listener: (...args: TEventMap[K]) => void  
): void {  
    if (!this.listeners[event]) {  
        this.listeners[event] = [];  
    }  
    this.listeners[event]!.push(listener);  
}
```

// Remove event listener

```
off<K extends keyof TEventMap>(  
    event: K,  
    listener: (...args: TEventMap[K]) => void  
): void {  
    if (!this.listeners[event]) return;  
  
    const index = this.listeners[event]!.indexOf(listener);  
    if (index > -1) {  
        this.listeners[event]!.splice(index, 1);  
    }  
}
```

// Emit event

```
emit<K extends keyof TEventMap>(  
    event: K,  
    ...args: TEventMap[K]  
): void {  
    if (!this.listeners[event]) return;  
  
    this.listeners[event]!.forEach(listener => {  
        listener(...args);  
    })  
}
```

```

});
}

// Add one-time event listener
once<K extends keyof TEventMap>(
  event: K,
  listener: (...args: TEventMap[K]) => void
): void {
  const onceWrapper = (...args: TEventMap[K]) => {
    this.off(event, onceWrapper);
    listener(...args);
  };
  this.on(event, onceWrapper);
}

// Remove all listeners for an event
removeAllListeners<K extends keyof TEventMap>(event?: K): void {
  if (event) {
    delete this.listeners[event];
  } else {
    this.listeners = {};
  }
}

// Get listener count
listenerCount<K extends keyof TEventMap>(event: K): number {
  return this.listeners[event]?.length || 0;
}
}

// Define your event map
interface MyAppEvents {
  'user:login': [user: { id: number; name: string }];
  'user:logout': [userId: number];
  'order:created': [order: { id: string; total: number; items: string[] }];
  'error': [error: Error, context: string];
}

// Usage
const eventEmitter = new TypeSafeEventEmitter<MyAppEvents>();

// Type-safe event listeners
eventEmitter.on('user:login', (user) => {
  console.log(`User ${user.name} logged in`);
  // user is correctly typed as { id: number; name: string }
});

```

```

eventEmitter.on('order:created', (order) => {
  console.log(`Order ${order.id} created with total ${order.total}`);
  // order is correctly typed
});

eventEmitter.on('error', (error, context) => {
  console.error(`Error in ${context}:`, error.message);
  // Both parameters are correctly typed
});

// Type-safe event emission
eventEmitter.emit('user:login', { id: 1, name: 'John Doe' });
eventEmitter.emit('order:created', {
  id: 'order-123',
  total: 99.99,
  items: ['item1', 'item2']
});

// TypeScript will catch errors:
// eventEmitter.emit('user:login', { id: 1 }); // Error: missing 'name'
// eventEmitter.emit('invalid:event', 'data'); // Error: event doesn't exist

```

12. Implement a Deep Clone Function with Type Safety

Answer:

typescript

// Type for cloneable values

```
type Cloneable =  
  | string  
  | number  
  | boolean  
  | null  
  | undefined  
  | Date  
  | RegExp  
  | Array<Cloneable>  
  | { [key: string]: Cloneable };
```

// Deep clone function

```
function deepClone<T extends Cloneable>(obj: T): T {  
  // Handle primitive types and null/undefined  
  if (obj === null || typeof obj !== 'object') {  
    return obj;  
  }  
  
  // Handle Date  
  if (obj instanceof Date) {  
    return new Date(obj.getTime()) as T;  
  }  
  
  // Handle RegExp  
  if (obj instanceof RegExp) {  
    return new RegExp(obj.source, obj.flags) as T;  
  }  
  
  // Handle Arrays  
  if (Array.isArray(obj)) {  
    return obj.map(item => deepClone(item)) as T;  
  }  
  
  // Handle Objects  
  const cloned = {} as T;  
  for (const key in obj) {  
    if (obj.hasOwnProperty(key)) {  
      cloned[key] = deepClone(obj[key]);  
    }  
  }  
  
  return cloned;  
}
```

// Advanced version with circular reference detection

```
function deepCloneWithCircularCheck<T extends Cloneable>(
  obj: T,
  visited = new WeakMap()
): T {
  // Handle primitive types and null/undefined
  if (obj === null || typeof obj !== 'object') {
    return obj;
  }

  // Check for circular references
  if (visited.has(obj)) {
    throw new Error('Circular reference detected');
  }

  // Handle Date
  if (obj instanceof Date) {
    return new Date(obj.getTime()) as T;
  }

  // Handle RegExp
  if (obj instanceof RegExp) {
    return new RegExp(obj.source, obj.flags) as T;
  }

  // Mark as visited
  visited.set(obj, true);

  let cloned: T;

  // Handle Arrays
  if (Array.isArray(obj)) {
    cloned = obj.map(item => deepCloneWithCircularCheck(item, visited)) as T;
  } else {
    // Handle Objects
    cloned = {} as T;
    for (const key in obj) {
      if (obj.hasOwnProperty(key)) {
        cloned[key] = deepCloneWithCircularCheck(obj[key], visited);
      }
    }
  }

  // Remove from visited after processing
  visited.delete(obj);

  return cloned;
}
```

```
}
```

```
// Type-safe version that preserves exact types
```

```
type DeepClone<T> = T extends string | number | boolean | null | undefined
```

```
  ? T
```

```
  : T extends Date
```

```
  ? Date
```

```
  : T extends RegExp
```

```
  ? RegExp
```

```
  : T extends (infer U)[]
```

```
  ? DeepClone<U>[]
```

```
  : T extends object
```

```
  ? { [K in keyof T]: DeepClone<T[K]> }
```

```
  : T;
```

```
function typedDeepClone<T>(obj: T): DeepClone<T> {
```

```
  return deepClone(obj as any) as DeepClone<T>;
```

```
}
```

```
// Usage examples
```

```
interface User {
```

```
  id: number;
```

```
  name: string;
```

```
  profile: {
```

```
    email: string;
```

```
    settings: {
```

```
      theme: 'light' | 'dark';
```

```
      notifications: boolean;
```

```
    };
```

```
  };
```

```
  tags: string[];
```

```
  createdAt: Date;
```

```
}
```

```
const originalUser: User = {
```

```
  id: 1,
```

```
  name: "John Doe",
```

```
  profile: {
```

```
    email: "john@example.com",
```

```
    settings: {
```

```
      theme: 'dark',
```

```
      notifications: true
```

```
    }
```

```
  },
```

```
  tags: ["admin", "premium"],
```

```
  createdAt: new Date()
```

```
};

// Clone with full type safety
const clonedUser = typedDeepClone(originalUser);
// clonedUser has exact type User, not just Cloneable

// Modify clone without affecting original
clonedUser.profile.settings.theme = 'light';
clonedUser.tags.push('new-tag');

console.log('Original user theme:', originalUser.profile.settings.theme); // 'dark'
console.log('Cloned user theme:', clonedUser.profile.settings.theme);    // 'light'

console.log('Original tags length:', originalUser.tags.length);          // 2
console.log('Cloned tags length:', clonedUser.tags.length);              // 3
```

Configuration & Tooling

13. Explain TypeScript Configuration (tsconfig.json)

Answer:

json

```
{
  "compilerOptions": {
    // Basic Options
    "target": "ES2020",           // ECMAScript target version
    "module": "commonjs",        // Module system
    "lib": ["ES2020", "DOM"],     // Library files to include
    "outDir": "./dist",          // Output directory
    "rootDir": "./src",          // Root of input files

    // Strict Type-Checking Options
    "strict": true,              // Enable all strict type-checking options
    "noImplicitAny": true,       // Error on expressions with implied 'any'
    "strictNullChecks": true,    // Enable strict null checks
    "strictFunctionTypes": true,  // Enable strict checking of function types
    "strictBindCallApply": true, // Enable strict 'bind', 'call', 'apply'
    "strictPropertyInitialization": true, // Ensure properties are initialized
    "noImplicitReturns": true,    // Error when not all paths return value
    "noImplicitThis": true,       // Error on 'this' with implied 'any'
    "noUnusedLocals": true,       // Error on unused local variables
    "noUnusedParameters": true,  // Error on unused parameters

    // Module Resolution Options
    "moduleResolution": "node",   // Module resolution strategy
    "baseUrl": ".",              // Base directory for non-relative modules
    "paths": {                   // Path mapping for module names
      "@/*": ["src/*"],
      "@components/*": ["src/components/*"],
      "@utils/*": ["src/utils/*"]
    },
    "esModuleInterop": true,       // Enable interop between CommonJS and ES modules
    "allowSyntheticDefaultImports": true, // Allow default imports from modules
    "resolveJsonModule": true,     // Include modules imported with .json extension

    // Emit Options
    "declaration": true,          // Generate .d.ts files
    "declarationMap": true,       // Generate sourcemap for .d.ts files
    "sourceMap": true,            // Generate .map files
    "removeComments": true,       // Remove comments in output
    "noEmitOnError": true,        // Don't emit if there are errors

    // Advanced Options
    "skipLibCheck": true,         // Skip type checking of declaration files
    "forceConsistentCasingInFileNames": true, // Ensure consistent casing
    "experimentalDecorators": true, // Enable experimental decorators
    "emitDecoratorMetadata": true, // Emit type metadata for decorators
  }
}
```

```

    "emitDecoratorMetadata": true // Emit metadata for decorators
  },
  "include": [
    "src/**/*",
    "tests/**/*"
  ],
  "exclude": [
    "node_modules",
    "dist",
    "**/*.spec.ts"
  ]
}

```

14. How do you set up TypeScript with different build tools?

Answer:

With Webpack:

```

javascript

// webpack.config.js
const path = require('path');

module.exports = {
  entry: './src/index.ts',
  module: {
    rules: [
      {
        test: /\.tsx?$/,
        use: 'ts-loader',
        exclude: /node_modules/,
      },
    ],
  },
  resolve: {
    extensions: ['.tsx', '.ts', '.js'],
    alias: {
      '@': path.resolve(__dirname, 'src'),
    },
  },
  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, 'dist'),
  },
};

```

With Vite:

typescript

// vite.config.ts

import { defineConfig } from 'vite';

import { resolve } from 'path';

export default defineConfig({

resolve: {

alias: {

'@': resolve(__dirname, 'src'),

},

},

build: {

lib: {

entry: resolve(__dirname, 'src/index.ts'),

name: 'MyLib',

fileName: (format) => `my-lib.\${format}.js`,

},

},

});

Package.json scripts:

json

{

"scripts": {

"build": "tsc",

"build:watch": "tsc --watch",

"dev": "ts-node src/index.ts",

"test": "jest",

"lint": "eslint src/**/*.ts",

"type-check": "tsc --noEmit"

}

}

Bonus: Advanced Patterns

15. Implement a Type-Safe State Machine

Answer:

typescript

// State machine type definitions

```
type StateMachineConfig<
  TState extends string,
  TEvent extends string,
  TContext = any
> = {
  initial: TState;
  context?: TContext;
  states: {
    [K in TState]: {
      on?: {
        [E in TEvent]?: {
          target: TState;
          action?: (context: TContext, event: { type: E; [key: string]: any }) => TContext;
        };
      };
      entry?: (context: TContext) => TContext;
      exit?: (context: TContext) => TContext;
    };
  };
};
```

// State machine implementation

```
class StateMachine<
  TState extends string,
  TEvent extends string,
  TContext = any
> {
  private currentState: TState;
  private context: TContext;
  private config: StateMachineConfig<TState, TEvent, TContext>;

  constructor(config: StateMachineConfig<TState, TEvent, TContext>) {
    this.config = config;
    this.currentState = config.initial;
    this.context = config.context as TContext;

    // Run entry action for initial state
    const initialStateConfig = this.config.states[this.currentState];
    if (initialStateConfig.entry) {
      this.context = initialStateConfig.entry(this.context);
    }
  }
}
```

```

send(event: { type: TEvent; [key: string]: any }): boolean {
    const currentStateConfig = this.config.states[this.currentState];
    const transition = currentStateConfig.on?.[event.type];

    if (!transition) {
        return false; // No transition defined for this event
    }

    // Run exit action for current state
    if (currentStateConfig.exit) {
        this.context = currentStateConfig.exit(this.context);
    }

    // Run transition action
    if (transition.action) {
        this.context = transition.action(this.context, event);
    }

    // Change state
    this.currentState = transition.target;

    // Run entry action for new state
    const newStateConfig = this.config.states[this.currentState];
    if (newStateConfig.entry) {
        this.context = newStateConfig.entry(this.context);
    }

    return true;
}

getState(): TState {
    return this.currentState;
}

getContext(): TContext {
    return this.context;
}
}

// Usage Example: Traffic Light State Machine
type TrafficLightStates = 'red' | 'yellow' | 'green';
type TrafficLightEvents = 'TIMER' | 'EMERGENCY';

interface TrafficLightContext {
    timer: number;
    emergencyActive: boolean;
}

```

```

const trafficLightConfig: StateMachineConfig <
  TrafficLightStates,
  TrafficLightEvents,
  TrafficLightContext
> = {
  initial: 'red',
  context: {
    timer: 0,
    emergencyActive: false
  },
  states: {
    red: {
      entry: (context) => ({ ...context, timer: 30 }),
      on: {
        TIMER: { target: 'green' },
        EMERGENCY: { target: 'red' }
      }
    },
    yellow: {
      entry: (context) => ({ ...context, timer: 5 }),
      on: {
        TIMER: { target: 'red' },
        EMERGENCY: { target: 'red' }
      }
    },
    green: {
      entry: (context) => ({ ...context, timer: 45 }),
      on: {
        TIMER: { target: 'yellow' },
        EMERGENCY: {
          target: 'red',
          action: (context) => ({ ...context, emergencyActive: true })
        }
      }
    }
  }
};

```

```

const trafficLight = new StateMachine(trafficLightConfig);

```

```

console.log(trafficLight.getState()); // 'red'
trafficLight.send({ type: 'TIMER' });
console.log(trafficLight.getState()); // 'green'
trafficLight.send({ type: 'EMERGENCY' });
console.log(trafficLight.getState()); // 'red'

```

```
console.log(trafficLight.getContext().emergencyActive); // true
```